ImmobiliaRende
Relazione sul progetto di piattaforma web per annunci immobiliari

Alessandro Taverniti — 245930

Anno accademico 2025/26 — Università della Calabria
Corso: Web Applications

Sommario

ImmobiliaRende è una piattaforma web per la pubblicazione e la consultazione di annunci immobiliari nella zona di Rende (Cosenza). Il sistema separa un *frontend* single-page in Angular da un *backend* REST in Spring Boot, con persistenza relazionale su PostgreSQL e autenticazione basata su JSON Web Token (JWT). Il documento descrive obiettivi, architettura, stack tecnologico e le principali funzionalità per visitatori, venditori, acquirenti e amministratori.

Indice

1 Introduzione e contesto **2**

2 Obiettivi **2**

3 Architettura generale **2**

4 Stack tecnologico **2**

5 Backend (Spring Boot) **2**
5.1 Organizzazione del codice 2
5.2 Sicurezza 3
5.3 Dati e configurazione 3

6 Frontend (Angular) **3**
6.1 Routing e caricamento lazy 3
6.2 Comunicazione con il backend 4
6.3 Interfaccia utente 4

7 Funzionalità per ruolo **4**
7.1 Visitatori e utenti autenticati (catalogo) 4
7.2 Venditore 4
7.3 Acquirente 4
7.4 Amministratore 4

8 Funzionalità trasversali **4**

9 Distribuzione e demo **4**

10 Conclusioni **5**

1 Introduzione e contesto

Il progetto nasce nell'ambito del corso *Web Applications* e implementa una applicazione completa di tipo *client-server*: il browser esegue un'interfaccia ricca (SPA) che comunica tramite HTTP/JSON con API REST; il server applica regole di business, validazione, sicurezza e accesso ai dati.

Il dominio è quello degli annunci immobiliari localizzati a Rende e dintorni: il nome del prodotto richiama in modo ironico l'unione tra "immobiliare" e il toponimo "Rende".

2 Obiettivi

Gli obiettivi principali sono:

- offrire un catalogo pubblico consultabile con filtri e mappa degli immobili;
- consentire ai *venditori* di gestire annunci, foto, ribassi di prezzo, aste e messaggi ricevuti;
- consentire agli *acquirenti* di cercare, contattare i venditori, partecipare ad aste, richiedere visite e recensire dopo una visita completata;
- fornire un *pannello amministrativo* per statistiche, moderazione (annunci, recensioni, aste) e gestione utenti (ban, promozione ad admin).

3 Architettura generale

L'applicazione segue un'architettura a strati:

- **Presentation layer (client)**: Angular con routing, componenti lazy-loaded, Bootstrap per il layout.
- **API layer (server)**: controller Spring che espongono risorse sotto il prefisso `/api`.
- **Service layer**: logica applicativa (regole su aste, ribassi, geocoding, costruzione URL foto, ecc.).
- **Persistence layer**: entità JPA, repository Spring Data, database PostgreSQL.

La comunicazione tra client e server è *stateless* dal punto di vista delle sessioni HTTP: dopo il login, il client allega un JWT alle richieste autenticate (tipicamente tramite header `Authorization`).

4 Stack tecnologico

La Tabella 1 riassume le tecnologie impiegate.

5 Backend (Spring Boot)

5.1 Organizzazione del codice

Il backend è organizzato in package tipici di Spring:

- `controller`: endpoint REST (`/api/auth`, `/api/annunci`, `/api/foto`, `/api/messaggi`, `/api/appuntamenti`, `/api/recensioni`, `/api/aste`, `/api/admin`, ecc.);
- `model`: entità JPA;

Tabella 1: Stack tecnologico del progetto.

Parte	**Tecnologia**
Frontend	Angular ∼21, TypeScript ∼5.9, Bootstrap 5.3
Backend	Java 17, Spring Boot 3.5.12
Persistenza	Spring Data JPA (Hibernate), PostgreSQL
Sicurezza	Spring Security, JWT (jjwt 0.12.6)
Mappe	Leaflet 1.9, embed Google Maps dove usato in UI
Grafici (admin)	Chart.js 4.x
Geocoding	Nominatim / OpenStreetMap (lato server)
Foto	Upload multipart, Thumbnailator (ridimensionamento), bytes in DB
Build / tool	Maven (backend), npm / Angular CLI (frontend)

- `dao`: repository Spring Data;
- `dto`: oggetti di trasferimento per richieste e risposta, con validazione Bean Validation (`@Valid`) dove previsto;
- `security`: configurazione CORS, filtro JWT, utilità per token, `UserDetailsService`;
- `service`: servizi di dominio (geocoding, account, ribasso annuncio, URL pubblici delle foto, ecc.);
- `proxy`: ad esempio `AnnuncioProxy` per letture aggregate senza problemi di lazy loading fuori transazione;
- `exception`: gestione centralizzata degli errori (risposte JSON uniformi).

5.2 Sicurezza

Spring Security protegge le API riservate. Il flusso JWT prevede login che restituisce un token; le richieste successive includono il token. I ruoli applicativi distingue tipicamente `VENDITORE`, `ACQUIRENTE` e `ADMIN`. La registrazione pubblica non consente di auto-assegnarsi il ruolo amministratore.

5.3 Dati e configurazione

La configurazione (`application.properties`) include URL del datasource PostgreSQL, secret per la firma JWT, URL pubblico del server (`app.server-url`) per costruire i link assoluti alle immagini servite da `/api/foto/{id}/file`, limiti multipart per upload. Con `spring.jpa.hibernate.ddl-auto` adeguato (es. `update` in sviluppo) Hibernate adatta lo schema; può essere presente `data.sql` per seed minimo (utente admin) e coerenza dati.

6 Frontend (Angular)

6.1 Routing e caricamento lazy

Le route principali includono reindirizzamento da "`/`" a `/home`, pagine pubbliche (`/home`, `/mappa`, `/login`, `/register`, `/annunci/:id`) e una sezione `/dashboard` con figli per `acquirente`, `venditore` e `admin`. I componenti delle dashboard sono protetti da `authGuard` e `roleGuard` (controllo ruolo nei `data` della route).

6.2 Comunicazione con il backend

Servizi Angular incapsulano le chiamate HTTP (`HttpClient`). Un interceptor può allegare il JWT alle richieste autenticate. In sviluppo il frontend punta tipicamente a `http://localhost:8080`; in produzione la base URL è configurabile tramite `environment`.

6.3 Interfaccia utente

Bootstrap 5 fornisce griglia, componenti e utilità responsive. Leaflet mostra la mappa globale degli annunci; Chart.js alimenta i grafici nella dashboard amministratore.

7 Funzionalità per ruolo

7.1 Visitatori e utenti autenticati (catalogo)

Elenco annunci con filtri (tipo, categoria) e ordinamento; dettaglio con galleria, mappa, recensioni ed eventuale sezione asta; mappa globale `/mappa`; registrazione come venditore o acquirente.

7.2 Venditore

CRUD annunci; upload foto con ottimizzazione lato server; geocoding dell'indirizzo; ribasso prezzo (prezzo precedente barrato in UI); creazione asta con scadenza; messaggi ricevuti per annuncio; condivisione link.

7.3 Acquirente

Ricerca e filtri; messaggi al venditore; offerte su asta aperta; una recensione per annuncio solo dopo visita in stato completato; dashboard personale (aste, messaggi, recensioni).

7.4 Amministratore

Statistiche con grafici; gestione utenti (ban/sban, promozione ad admin); moderazione annunci e recensioni; gestione aste; sulle richieste di visita l'admin può vedere l'elenco completo rispetto al venditore che vede solo le proprie.

8 Funzionalità trasversali

- **Geocoding**: il backend interroga Nominatim per risolvere indirizzi in coordinate, usate nelle mappe.
- **Foto**: upload con validazione tipo e dimensione; Thumbnailator ridimensiona (es. fino a 1920×1080, JPEG con qualità indicata); persistenza come `bytea`; risposte HTTP con cache dove applicabile; URL nelle risposte JSON tramite servizio dedicato e base URL configurata.
- **Test backend**: suite JUnit su utilità JWT, servizi, proxy annuncio, gestione errori, ecc. (`mvn test`).

9 Distribuzione e demo

Il frontend è distribuito come sito statico (output di `ng build`) su Vercel: le route della SPA sono reindirizzate verso `index.html` tramite le regole definite in `vercel.json`, così il routing lato client di Angular funziona anche aggiornando la pagina o aprendo URL profondi. La demo pubblica dell'interfaccia è raggiungibile all'indirizzo:

https://webapp-immobiliarende.vercel.app/home

Il **backend** Spring Boot e il **database** PostgreSQL sono ospitati su **Railway**: l'applicazione Java viene eseguita come servizio deployabile sul cloud Railway, mentre il database relazionale è fornito come servizio PostgreSQL gestito sulla stessa piattaforma (o collegato tramite variabili d'ambiente, URL JDBC e credenziali configurate lato server). In questo modo API, persistenza e sicurezza JWT restano disponibili su Internet; il frontend su Vercel punta alla base URL pubblica del backend Railway definita negli `environment` di produzione del build Angular, in modo che catalogo, autenticazione e dashboard dialoghino con l'istanza remota e non solo con `localhost`.

10 Conclusioni

ImmobiliaRende integra in un unico progetto competenze su API REST, sicurezza JWT, modellazione relazionale, SPA moderna con routing protetto e integrazione di servizi esterni (geocoding, mappe). L'architettura modulare consente di estendere il dominio (nuovi filtri, notifiche, pagamenti) mantenendo separazione tra contratti REST, logica di servizio e presentazione.